

Experiment 6

Student Name: Prateek Verma
Branch: MCA (CC & DEVOPS)
Semester: IInd
Subject Name: Technical Training

UID: 25MCC20062
Section/Group: 25MCC-101/A
Date of Performance: 24/02/2026
Subject Code: 25CAP-652

Title: Views in SQL Databases

1. Aim:

- a. Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

2. Software Requirements:

- a. PostgreSQL (Database Server)
- b. pgAdmin
- c. OS: Windows

3. Objective:

- a. Data Abstraction: To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- b. Enhanced Security: To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- c. Query Simplification: To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- d. View Management: To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

4. Theory: A View is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".

- a. Simple Views: Created from a single table without any aggregate functions or grouping. These are often updatable.
- b. Complex Views: Created from multiple tables using JOINS, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.
- c. Security Layer: In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.

- d. Benefits: They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

5. Procedure of the Practical:

- Start the system.
- Open pgAdmin.
- Create and select the database in which you want to perform the experiment.
- Establish connection to the database using Alt+Shift+Q.
- Run the following queries given in Experiment Steps (6).

6. Experiment Steps:

Base Table:

Employees

	emp_id [PK] integer	name character varying (100)	role character varying (50)	salary integer	status character varying (20)	dept_id integer	hire_date date
1	101	Anay Verma	Senior Dev	95000	Active	1	2025-01-10
2	102	Bhavna Singh	Manager	82000	Active	2	2024-05-15
3	103	Chaitanya K.	Intern	35000	Inactive	1	2026-02-01
4	104	Deepak Raj	Analyst	60000	Active	3	2023-11-20
5	105	Esha Gupta	Lead Designer	88000	Active	2	2024-08-12
6	106	Fahad Khan	Sales Exec	55000	Active	4	2025-03-01
7	107	Gauri Mehta	DevOps	92000	Active	1	2024-12-05
8	108	Hitesh Shah	Recruiter	48000	Inactive	3	2023-09-18

Department

	dept_id [PK] integer	dept_name character varying (50)	location character varying (50)	budget integer
1	1	Engineering	San Francisco	500000
2	2	Marketing	New York	250000
3	3	HR	Chicago	150000
4	4	Sales	Austin	300000

- Creating a simple view to list active employees without exposing entire table.

CREATE VIEW Active_Employees_List AS

SELECT emp_id, name, role, dept_id FROM Employees
WHERE status = 'Active';

	emp_id integer	name character varying (100)	role character varying (50)	dept_id integer
1	101	Anay Verma	Senior Dev	1
2	102	Bhavna Singh	Manager	2
3	104	Deepak Raj	Analyst	3
4	105	Esha Gupta	Lead Designer	2
5	106	Fahad Khan	Sales Exec	4
6	107	Gauri Mehta	DevOps	1

- b. Creating a view for joining multiple tables to simplify the retrieval of data distributed across employees and departments table.

CREATE VIEW Employee_Department_Details AS

SELECT e.emp_id, e.name AS employee_name, e.role, d.dept_name, d.location,
e.salary FROM Employees e

JOIN Departments d ON e.dept_id = d.dept_id;

	emp_id integer	employee_name character varying (100)	role character varying (50)	dept_name character varying (50)	location character varying (50)	salary integer
1	101	Anay Verma	Senior Dev	Engineering	San Francisco	95000
2	102	Bhavna Singh	Manager	Marketing	New York	82000
3	103	Chaitanya K.	Intern	Engineering	San Francisco	35000
4	104	Deepak Raj	Analyst	HR	Chicago	60000
5	105	Esha Gupta	Lead Designer	Marketing	New York	88000
6	106	Fahad Khan	Sales Exec	Sales	Austin	55000
7	107	Gauri Mehta	DevOps	Engineering	San Francisco	92000
8	108	Hitesh Shah	Recruiter	HR	Chicago	48000

- c. Creating an advanced view to provide department wise statistics.

CREATE VIEW Department_Summary AS

SELECT d.dept_name, d.location, COUNT(e.emp_id) AS total_employees,
SUM(e.salary) AS total_salary_payout FROM Departments d

LEFT JOIN Employees e ON d.dept_id = e.dept_id

GROUP BY d.dept_id, d.dept_name, d.location, d.budget;

	dept_name character varying (50)	location character varying (50)	total_employees bigint	total_salary_payout bigint
1	Engineering	San Francisco	3	222000
2	Marketing	New York	2	170000
3	HR	Chicago	2	108000
4	Sales	Austin	1	55000

7. Input/Output Details:

- The input for the experiment is the SQL queries mentioned in Experiment Steps (6).
- The output for each query is attached after the query itself.

8. Learning Outcome

- Abstraction Proficiency: create and query views to simplify efficient data access and abstraction.
- Security Implementation: how to use views for data masking and providing restricted access to sensitive information.
- Syntactic Accuracy: gaining knowledge of correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- Real-world Application: design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.